

Seeing and Reflecting: Multimodal Memory-Enhanced Agent Collaboration for Recommendation

Hao Cong^{1,*}, Huizu Lin^{2,*}, Zihan Wang^{3,*}, Chengkai Huang^{4,5,#},
Quan Z. Sheng⁵, Lina Yao^{4,6}

¹Tsinghua University, ²University of Science and Technology of China,
³Peking University, ⁴The University of New South Wales,
⁵Macquarie University, ⁶CSIRO's Data61

*Equal contribution. #Corresponding author.

Correspondence: chengkai.huang1@unsw.edu.au

Abstract

Large language model (LLM)-based agentic recommender systems show promise in modeling user preferences through natural-language reasoning, yet they remain limited by text-centric inputs and coarse-grained memory updates, making agents prone to missing visual evidence, semantic noise, and preference drift. To address these limitations, we propose **MMEACR**, a **Multimodal Memory-Enhanced Agent Collaboration** framework for recommendation. MMEACR introduces a dual-track memory architecture that separates interpretable agent reasoning from fine-grained multimodal matching. In the reasoning track, collaborative User and Item Memory Agents maintain persistent multimodal memories and update them through an attribute-guided reinforcement-and-reflection mechanism. In the matching track, a decoupled multimodal embedding memory is built from raw interaction narratives and item images to preserve detailed cross-modal signals beyond structured memory updates. The two tracks are integrated through weighted Reciprocal Rank Fusion to produce robust and interpretable rankings. Experiments on three real-world domains show that MMEACR achieves strong overall performance against competitive LLM-based and agent-based baselines, with notable gains in visually grounded recommendation scenarios.

1 Introduction

Large language models (LLMs) have recently enabled a new class of agentic recommender systems that model user preferences through natural-language reasoning and interaction (Huang et al., 2025b, 2024a, 2025a). Representative methods such as AgentCF (Zhang et al., 2024) formulate recommendation as a collaborative process among language agents, where agents compare candidate

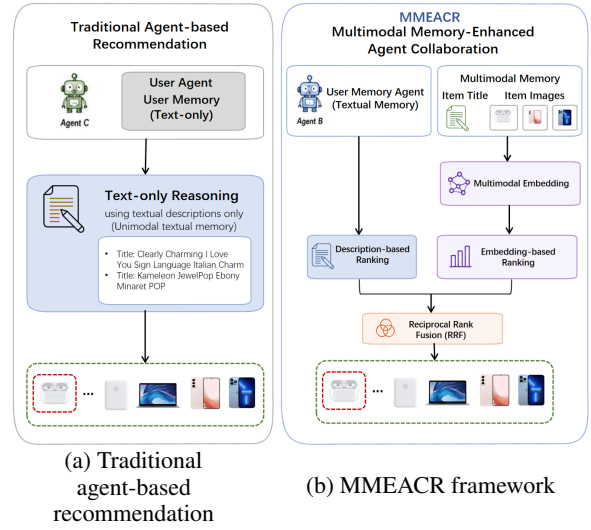


Figure 1: Comparison between conventional text-centric agent-based recommendation and MMEACR. MMEACR combines multimodal agent memory evolution with embedding-based cross-modal matching and fuses their rankings via Reciprocal Rank Fusion (RRF).

items, generate rationales, and update their internal memories. This paradigm improves interpretability and flexibility by externalizing preference modeling into language-mediated reasoning rather than relying solely on gradient-based representation learning.

Despite their promise, existing LLM-based recommendation agents still suffer from two key limitations. First, most methods remain text-centric, constructing user and item memories mainly from textual histories, reviews, or metadata while underusing visual evidence such as product images (Liu et al., 2025). This restricts their ability to model preferences in visually grounded domains such as fashion, electronics, and media products, where appearance, style, packaging, and other visual attributes often play an important role. Second, cur-

rent memory update mechanisms are often coarse-grained. Agents typically revise their profiles through free-form reflection or direct history accumulation, which can introduce redundant descriptions, amplify spurious rationales, and cause preference drift over repeated interactions. These issues become more challenging when textual and visual signals must be jointly interpreted and selectively consolidated.

To address these challenges, we propose **MMEACR**, a **M**ultimodal **M**emory-**E**nhanced **A**gent **C**ollaboration framework for recommendation. MMEACR is motivated by the idea of *Seeing and Reflecting*. *Seeing* grounds agents in multimodal evidence by initializing and enriching item memories with both textual metadata and image-derived descriptions. *Reflecting* enables structured memory evolution by reinforcing aligned preference signals and correcting misleading ones after user-item interactions.

MMEACR contains two complementary tracks. The reasoning track uses cooperative User and Item Memory Agents to maintain persistent language memories. During interaction, an LLM-based agent compares candidate items, generates a rationale, and updates memories according to whether the predicted preference matches the observed feedback. To reduce noisy updates, MMEACR extracts preference-relevant attributes from a predefined semantic attribute space and uses them to guide both reinforcement and reflection. This helps preserve stable user interests while revising inaccurate or drifting memory descriptions.

The matching track complements language-based reasoning with dense multimodal representations. Since structured memory updates may filter out fine-grained details, MMEACR also maintains raw interaction narratives and item images for embedding-based matching. A multimodal embedding module encodes these signals into dense representations, capturing cross-modal semantics that may be difficult to express in concise textual memories. Finally, MMEACR combines the reasoning-based and embedding-based rankings through weighted Reciprocal Rank Fusion, allowing the final recommendation to benefit from both interpretable agent reasoning and fine-grained multimodal similarity.

Our contributions are summarized as follows:

- We propose **MMEACR**, a multimodal memory-enhanced agent collaboration frame-

work that grounds LLM-based recommendation agents in both textual and visual evidence.

- We design an attribute-guided memory evolution mechanism that enables User and Item Memory Agents to reinforce aligned preferences and reflect on misaligned ones, reducing semantic noise and preference drift.
- We introduce a dual-track recommendation pipeline that combines interpretable language-agent reasoning with multimodal embedding-based matching through Reciprocal Rank Fusion, achieving strong performance across multiple domains.

2 Related Work

2.1 LLM-based Recommendation

Large language models (LLMs) have recently been explored for recommendation due to their strong abilities in semantic understanding, instruction following, and natural-language reasoning (Hou et al., 2024a; Zhao et al., 2024; Gao et al., 2026). Early LLM-based recommenders mainly use frozen LLMs as zero-shot or few-shot rankers, where user histories and candidate items are converted into textual prompts for preference prediction (Hou et al., 2024a). Other studies further improve task adaptation through Chain-of-Thought reasoning, instruction tuning, or retrieval-augmented generation, enabling LLMs to better align textual semantics with collaborative signals (Wei et al., 2022; Bao et al., 2023; Shi et al., 2025). More recently, agentic recommender systems have represented users and items as autonomous language agents that interact, exchange feedback, and update their memories during recommendation (Zhang et al., 2024; Liu et al., 2025). These methods improve interpretability by making preference modeling explicit in language-based reasoning processes. Meanwhile, multimodal LLMs and multimodal embedding models have shown the potential to incorporate visual and textual signals for richer item understanding (Lyu et al., 2023; Zhang et al., 2025a). However, most existing agentic recommenders still rely primarily on textual memories or directly inject multimodal information into prompts, which may lead to verbose contexts, redundant semantics, and unstable reasoning. In contrast, MMEACR adopts a dual-track design that separates structured agentic reasoning from dense multimodal matching, allowing the framework to use visual evidence

without overloading the language-agent memory.

2.2 Memory in LLM Agents

Memory is essential for LLM agents to maintain consistency, accumulate experience, and support reflection across interactions (Ferrag et al., 2025; Zhang et al., 2025b; Ye et al., 2026). Existing agent memory mechanisms range from short-term context buffers to long-term episodic stores, retrieval-based memories, and reflection-based memory management (Tang et al., 2026; Jiao et al., 2026a,b). Although these mechanisms have been widely studied in general-purpose agent systems, applying them to recommendations remains challenging. User preferences are often dynamic, sparse, and attribute-dependent, while item descriptions may contain noisy or redundant information. Simply appending interaction histories or retrieving past episodes can therefore introduce irrelevant details, amplify spurious preference signals, and cause preference drift over time (Salve et al., 2024; Li et al., 2025). Recent memory-enhanced recommender agents attempt to update user and item profiles through interaction feedback, but their updates are often coarse-grained and lack explicit constraints on which semantic attributes should be reinforced or corrected. MMEACR addresses this limitation with an attribute-guided memory evolution mechanism, where preference-relevant attributes are extracted from user-item comparisons and used to guide reinforcement and reflection. This design enables agents to consolidate stable preference signals while reducing semantic noise in long-term multimodal memory.

3 Problem Formulation

Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively. For each user $u \in \mathcal{U}$, we assume a chronologically ordered interaction history that reflects the user’s past preferences. Given a candidate item set $\mathcal{C} = \{c_1, \dots, c_n\} \subseteq \mathcal{I}$, the goal is to generate a personalized ranking of these candidates according to the user’s preference. Formally, we define the recommendation function as:

$$\hat{\mathcal{C}} = f_{\text{LLM}}(u, \mathcal{C}), \quad (1)$$

where $\hat{\mathcal{C}}$ denotes the ranked candidate list produced by the LLM-based recommendation framework.

Memory in Agents. Following the agent-based recommendation paradigm (Zhang et al., 2024), we model the collaborative recommendation process

with two types of LLM-powered memory agents: a **User Memory Agent (UA)** and an **Item Memory Agent (IA)**. Rather than relying on static textual profiles or fixed representation embeddings, MMEACR treats memories as persistent semantic states that can be updated through interactions. Specifically, the user memory M_u is an evolving textual narrative that summarizes the user’s long-term preferences, behavioral patterns, and fine-grained attribute tendencies. The item memory M_i describes item i by integrating its textual metadata with visual descriptions derived from product images. During recommendation and memory evolution, the User and Item Memory Agents collaborate through LLM-based reasoning over $\{M_u\} \cup \{M_i\}$, enabling the framework to capture preference-relevant semantic signals and support reasoning-aware user–item interactions.

4 Methodology

We present **MMEACR**, a multimodal memory-enhanced agent collaboration framework for recommendation. As shown in Figure 2, MMEACR contains two complementary tracks: (1) a *reasoning track*, where User and Item Memory Agents maintain and update interpretable multimodal memories through LLM-based interaction; and (2) a *matching track*, where raw interaction narratives and item images are encoded into dense multimodal embeddings. The two tracks are finally combined through weighted Reciprocal Rank Fusion (RRF) to produce the final recommendation ranking.

4.1 Multimodal Agent Memory

4.1.1 Memory Initialization

MMEACR first initializes semantic memories for users and items. For each item $i \in \mathcal{I}$, let T_i denote its textual title and $\mathcal{V}_i = \{v_{i,1}, \dots, v_{i,N_i}\}$ denote its associated images, where $N_i \leq 5$. To convert visual evidence into language-readable semantics, we use a multimodal LLM to generate an image-grounded description:

$$D_i = \text{MLLM}(T_i, \mathcal{V}_i). \quad (2)$$

The initial item memory is then constructed by integrating the title and visual description:

$$M_i^{(0)} = \text{LLM}(T_i, D_i). \quad (3)$$

Here, $M_i^{(0)}$ serves as a multimodal textual memory that summarizes the item’s key attributes. For

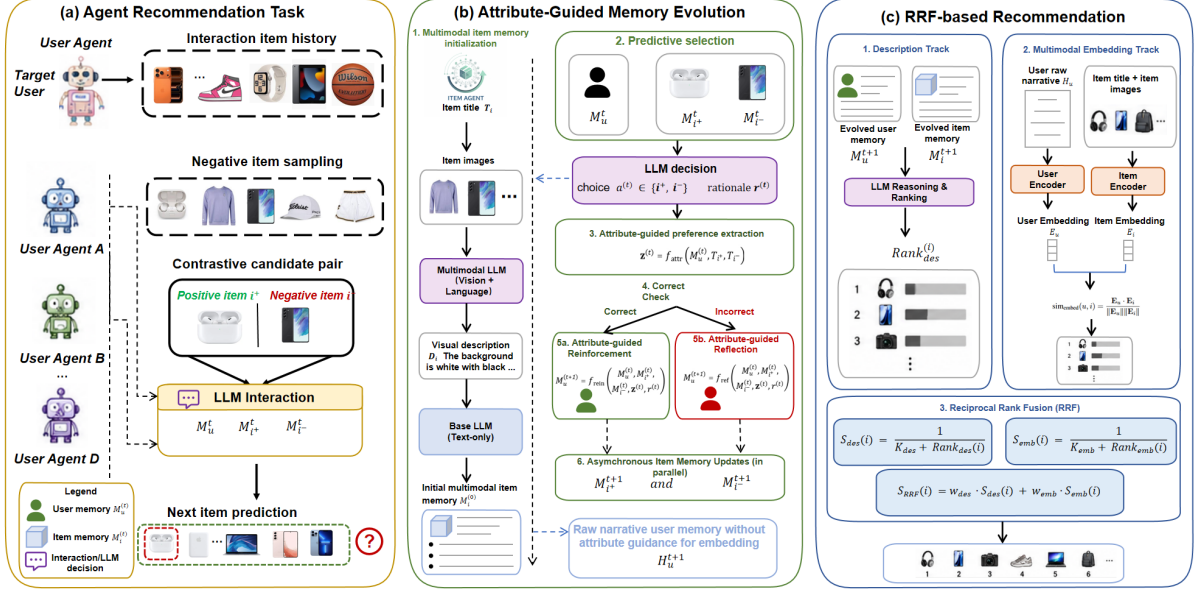


Figure 2: Overview of MMEACR. The reasoning track performs attribute-guided memory evolution with User and Item Memory Agents, while the matching track preserves fine-grained cross-modal signals through multimodal embedding memory. The final ranking is produced by fusing both tracks via RRF.

each user u , we initialize the user memory $M_u^{(0)}$ with a domain-specific template, such as a general preference statement for music, electronics, or fashion products. In addition to structured memories, we maintain raw narratives H_u and H_i for users and items, which preserve unfiltered interaction histories and multimodal descriptions for embedding-based matching.

4.1.2 Attribute-Guided Memory Evolution

The reasoning track updates agent memories through contrastive user-item interactions. At each interaction step t , we construct a triplet (u, i^+, i^-) , where i^+ is the ground-truth preferred item and i^- is a sampled negative item. Given the current user memory $M_u^{(t)}$ and item memories $M_{i^+}^{(t)}$ and $M_{i^-}^{(t)}$, the LLM-based interaction agent selects the item that better matches the user’s preference and generates a rationale:

$$a^{(t)}, r^{(t)} = f_{\text{dec}} \left(M_u^{(t)}, M_{i^+}^{(t)}, M_{i^-}^{(t)} \right), \quad (4)$$

where $a^{(t)} \in \{i^+, i^-\}$ is the selected item and $r^{(t)}$ is the natural-language rationale. The correctness of the selection is determined by:

$$y^{(t)} = \mathbb{I} \left[a^{(t)} = i^+ \right]. \quad (5)$$

To avoid updating memories with noisy free-form rationales, MMEACR introduces an attribute-guided preference extraction step. Let $\mathcal{A} =$

$\{a_1, \dots, a_K\}$ be a predefined semantic attribute space, where each attribute corresponds to a high-level preference factor such as style, functionality, portability, or visual appearance. Conditioned on the current memory, the contrastive item pair, and the generated rationale, the attribute extractor produces a structured preference signal:

$$\mathbf{z}^{(t)} = f_{\text{attr}} \left(M_u^{(t)}, T_{i^+}, T_{i^-}, r^{(t)}, \mathcal{A} \right), \quad (6)$$

where $\mathbf{z}^{(t)} \in \mathbb{R}^K$ encodes the attributes that explain the preference difference between i^+ and i^- .

The extracted attribute signal is then used to guide memory evolution. If the LLM selection is correct, MMEACR reinforces the current preference pattern; otherwise, it performs reflection to correct the misaligned memory:

$$\mathbf{x}_u^{(t)} = \left(M_u^{(t)}, M_{i^+}^{(t)}, M_{i^-}^{(t)}, \mathbf{z}^{(t)}, r^{(t)} \right),$$

$$M_u^{(t+1)} = \begin{cases} f_{\text{rein}} \left(\mathbf{x}_u^{(t)} \right), & y^{(t)} = 1, \\ f_{\text{refl}} \left(\mathbf{x}_u^{(t)} \right), & y^{(t)} = 0. \end{cases} \quad (7)$$

This design allows the User Memory Agent to consolidate stable interests while revising inaccurate or drifting preference descriptions.

The Item Memory Agents are updated asynchronously for the interacted positive and negative items:

$$M_{i^+}^{(t+1)} = f_{\text{pos}} \left(M_{i^+}^{(t)}, M_u^{(t)}, \mathbf{z}^{(t)}, y^{(t)} \right), \quad (8)$$

$$M_{i^-}^{(t+1)} = f_{\text{neg}} \left(M_{i^-}^{(t)}, M_u^{(t)}, \mathbf{z}^{(t)}, y^{(t)} \right). \quad (9)$$

Meanwhile, the raw user narrative is updated by appending the preferred item’s metadata and visual description:

$$H_u^{(t+1)} = H_u^{(t)} \oplus [T_{i+}; D_{i+}], \quad (10)$$

where $\oplus$ denotes chronological concatenation. Similarly, raw item narratives are maintained for the interacted items. These raw narratives are not filtered by the attribute space, ensuring that fine-grained cross-modal information remains available for the embedding-based matching track.

4.1.3 Reasoning-Based Ranking

After iterative memory evolution, the reasoning track ranks candidate items using the evolved user and item memories. Given a candidate set $\mathcal{C}$, the LLM-based ranker compares the user memory M_u with the candidate item memories $\{M_c \mid c \in \mathcal{C}\}$ and produces a description-based ranking:

$$\pi_{\text{des}} = \text{Rank}_{\text{LLM}}(M_u, \{M_c \mid c \in \mathcal{C}\}). \quad (11)$$

This ranking is interpretable because it is derived from explicit user and item memory descriptions.

4.2 Multimodal Embedding Memory

While the reasoning track provides interpretable preference reasoning, structured memory updates may filter out fine-grained visual or textual details. Therefore, MMEACR introduces a multimodal embedding memory to preserve raw cross-modal signals for dense matching.

4.2.1 Item Embedding Memory

For each item i , we use a pretrained multimodal embedding model MEM (Zhang et al., 2025a) to encode its title and images. The item embedding is computed by averaging image-title representations:

$$\mathbf{e}_i = \frac{1}{N_i} \sum_{j=1}^{N_i} \text{MEM}(T_i, v_{i,j}). \quad (12)$$

This embedding captures both textual and visual item semantics.

4.2.2 User Embedding Memory

For each user u , we construct the user embedding from the raw interaction narrative and the images of historically preferred items. Let $\mathcal{P}_u$ denote the set of items preferred by user u in the interaction history. The user embedding is computed as:

$$\mathbf{e}_u = \frac{1}{|\mathcal{P}_u|} \sum_{i \in \mathcal{P}_u} \text{MEM}(H_u, \bar{v}_i), \quad (13)$$

where $\bar{v}_i$ denotes the representative visual content of item i . Unlike the attribute-guided memory M_u , the raw narrative H_u preserves complete historical descriptions, allowing the embedding track to capture detailed semantic and visual preference signals.

4.2.3 Embedding-Based Ranking

The embedding track ranks candidate items by cosine similarity:

$$s_{\text{emb}}(u, i) = \frac{\mathbf{e}_u^\top \mathbf{e}_i}{\|\mathbf{e}_u\| \|\mathbf{e}_i\|}. \quad (14)$$

Sorting candidates by $s_{\text{emb}}(u, i)$ yields the embedding-based ranking π_{emb} .

4.3 Hybrid Ranking via Reciprocal Rank Fusion

The reasoning and matching tracks capture complementary signals. The reasoning track provides interpretable preference judgments based on evolved agent memories, while the embedding track captures fine-grained cross-modal similarity from raw narratives and images. To combine them, MMEACR adopts a weighted Reciprocal Rank Fusion strategy.

For each candidate item i , we first compute its rank-based scores from the two tracks:

$$S_{\text{des}}(i) = \frac{1}{k_{\text{des}} + \text{rank}_{\pi_{\text{des}}}(i)}, \quad (15)$$

$$S_{\text{emb}}(i) = \frac{1}{k_{\text{emb}} + \text{rank}_{\pi_{\text{emb}}}(i)}, \quad (16)$$

where $\text{rank}_{\pi_{\text{des}}}(i)$ and $\text{rank}_{\pi_{\text{emb}}}(i)$ denote the positions of item i in the reasoning-based and embedding-based rankings, respectively. The constants k_{des} and k_{emb} control the smoothness of rank contributions.

The final fusion score is:

$$S_{\text{RRF}}(i) = w_{\text{des}} S_{\text{des}}(i) + w_{\text{emb}} S_{\text{emb}}(i), \quad (17)$$

where w_{des} and w_{emb} balance the contributions of the two tracks. The final recommendation list is obtained by sorting all candidate items in descending order of $S_{\text{RRF}}(i)$.

5 Experiments

Datasets. Following previous work (Zhang et al., 2024; Huang et al., 2023b,a, 2024b), we conduct experiments on three text-intensive subsets of the

Table 1: Performance comparison on three domains. Best results are in **bold**, second-best are underlined. *Imp.* denotes the relative improvement of our method over the strongest baseline. All results are averaged over five runs with different seeds.

Method	CDs_and_Vinyl				Cell_Phones_and_Accessories				Fashion			
	N@1	N@5	N@10	MRR	N@1	N@5	N@10	MRR	N@1	N@5	N@10	MRR
Pop	0.1100	0.3237	0.4708	0.3138	0.1300	0.3173	0.4734	0.3177	0.1100	0.3241	0.4749	0.3180
BM25	0.1600	0.1769	0.4439	0.2852	0.2100	0.3691	0.4747	0.3741	0.2100	0.3708	0.4351	0.3650
SASRec	0.1400	0.3206	0.4790	0.3253	0.1200	0.3096	0.4610	0.3020	0.1300	0.2934	0.4609	0.3034
LLMSeqSim	0.1800	0.4157	0.5319	0.3897	0.2100	0.4267	0.5344	0.3938	0.1800	<u>0.4364</u>	<u>0.5390</u>	0.3982
MLLMSeqSim	0.1800	0.4541	0.5418	0.4005	0.1300	0.3226	0.4778	0.3234	0.1600	0.3844	<u>0.5067</u>	0.3573
LLMRank	0.1895	0.3628	0.5085	0.3624	0.2043	0.4135	0.5299	0.3888	<u>0.2200</u>	0.4014	0.5359	<u>0.3987</u>
AgentCF	0.1900	0.3941	0.5169	0.3731	0.2000	<u>0.4393</u>	<u>0.5496</u>	0.4121	0.1700	0.3399	0.4943	0.3447
CoTAgent	<u>0.2700</u>	0.5941	<u>0.6315</u>	<u>0.5127</u>	<u>0.2800</u>	0.4334	0.5339	<u>0.4213</u>	0.1900	0.3783	0.5081	0.3772
MMEACR-DES	0.3200	0.5157	0.6088	0.4900	0.3000	0.5066	0.5926	0.4692	0.2500	0.4641	0.5708	0.4407
MMEACR-EMB	0.2600	0.5298	0.6074	0.4847	0.2500	0.4979	0.5815	0.4517	0.3400	0.5276	0.6179	0.5019
MMEACR-RRF	0.3400	<u>0.5632</u>	0.6354	0.5224	0.3200	0.5327	0.6214	0.5049	0.3200	0.5382	0.6230	0.5069
Improv(%)	20.59%	-5.20%	0.62%	1.89%	14.3%	21.26%	13.06%	19.84%	45.45%	23.33%	15.58%	27.14%

Table 2: Dataset statistics.

Data	#Users	#Items	#Inter.	Sparsity
CDs	100	781	500	99.36%
Cell_Phones	100	753	500	99.34%
Fashion	100	763	500	99.34%

Amazon review dataset (CDs, Cell_phones, and Fashion).

Baselines. We evaluate our method against several representative baselines, including popularity-based **Pop**, text-matching **BM25** (Robertson and Zaragoza, 2009), self-attention sequential recommender **SASRec** (Kang and McAuley, 2018), LLM-based ranker **LLMRank** (Hou et al., 2024b), embedding-similarity models **LLMSeqSim** (Harte et al., 2023) and **MLLMSeqSim**, as well as the agent-based **AgentCF** (Zhang et al., 2024) and **CoTAgent** (Wei et al., 2022). Specifically, **CoTAgent** incorporates zero-shot Chain-of-Thought (CoT) prompting to facilitate transparent decision-making and produce interpretable reasoning rationales for each recommendation. Pop ranks items by interaction frequency, while BM25 retrieves candidates using textual similarity to user histories. LLM-Rank uses GPT-4o-mini as a zero-shot ranker conditioned on sequential histories. LLMSeqSim constructs a session representation via LLM-generated item embeddings and retrieves top-k similar items, whereas MLLMSeqSim extends this with multi-modal LLM embeddings. AgentCF treats users and items as autonomous LLM agents that interact and update memories to perform collaborative filtering. For our method **MMEACR**, we additionally include three variants. **MMEACR-DES** uses the user’s and item’s latest memories with a 1-positive-9-negative candidate set, prompting the LLM to return a ranking for computing NDCG

Table 3: Ablation study on Cell_phones and Fashion. Best results are in **bold**, and second-best results are underlined.

Cell_phones				
Variant	N@1	N@5	N@10	MRR
Full	.3000	.5066	<u>.5926</u>	<u>.4692</u>
w/o Attr.	<u>.2700</u>	.5066	.5987	.4749
w/o Refl. & Attr.	.2200	.4743	.5663	.4323
w/o User & Attr.	.1800	.4047	.5235	.3804
w/o Item & Attr.	.2500	<u>.5051</u>	.5800	.4500
Fashion				
Variant	N@1	N@5	N@10	MRR
Full	.2500	.4641	.5708	.4407
w/o Attr.	<u>.2300</u>	.4492	<u>.5605</u>	<u>.4277</u>
w/o Refl. & Attr.	<u>.2300</u>	<u>.4555</u>	.5585	.4245
w/o User & Attr.	.1500	.3245	.4826	.3305
w/o Item & Attr.	<u>.2300</u>	.4317	.5468	.4107

and MMR. **MMEACR-EMB** fuses the user’s five recent item-image embeddings with the latest memory via GME, combines item image–title embeddings, and ranks candidates using cosine similarity. **MMEACR-RRF** merges the rankings from MMEACR-DES and MMEACR-EMB using the RRF formula.

Evaluation Metrics. We adopt a leave-one-out strategy (Kang and McAuley, 2018): for each user, the last interaction is used for testing and the second-to-last for validation. Following previous works (Zhang et al., 2024), each evaluation case includes one positive item and 9 randomly sampled negatives from the same domain.

Quantative Analysis. Table 1 reports the overall comparison across three domains. Our proposed **MMEACR-RRF** consistently delivers the strongest or highly competitive performance across

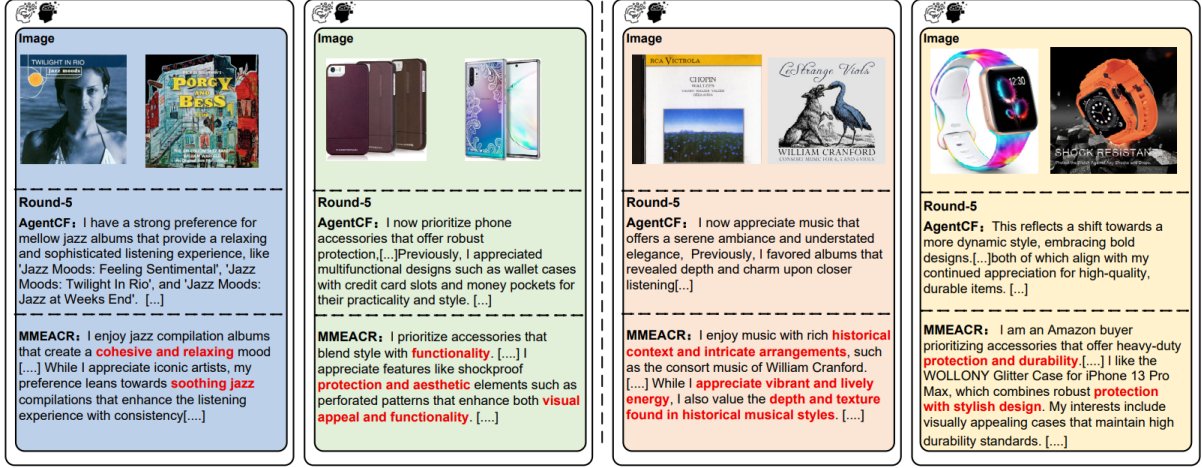


Figure 3: Case study of memory evolution and profile reflection at Round-5 across four semantic domains. Compared to AgentCF, our MMEACR demonstrates a superior capability to filter cross-modal noise and capture attribute-guided user preferences (highlighted in red) such as “cohesive and relaxing mood”, “protection and aesthetic”, and “historical context”, securing more precise long-term interest alignment.

datasets and metrics. On *CDs*, our method improves N@1 by 20.59% and MRR by 1.89% over the strongest baseline, while maintaining competitive performance on other metrics. The slightly lower NDCG@5 compared with CoTAgent may be attributed to the relatively low complexity of the CDs dataset, where explicit CoT reasoning is already sufficient to capture user preferences and rank relevant items effectively, leaving limited room for additional gains from attribute-guided reasoning and multimodal embedding fusion. On *Cell_Phones*, the improvements are consistent across all metrics, with gains of 14.3% on N@1 and 21.26% on N@5, demonstrating robust ranking quality in more attribute-diverse scenarios. Notably, on the visually intensive *Fashion* domain, our approach achieves substantial gains, improving N@1, N@5, and MRR by 45.45%, 23.33%, and 27.14%, respectively, highlighting the benefit of multimodal memory modeling in visually grounded recommendation.

Moreover, while both single-branch variants (MMEACR-DES and MMEACR-EMB) already achieve competitive performance, the fused model consistently yields further improvements, indicating that LLM-based reasoning and multimodal embedding similarity provide complementary signals. Compared with prior agent-based methods such as AgentCF and CoTAgent, our framework demonstrates consistent advantages across domains, validating the effectiveness of structured multimodal memory and reflective agent collabo-

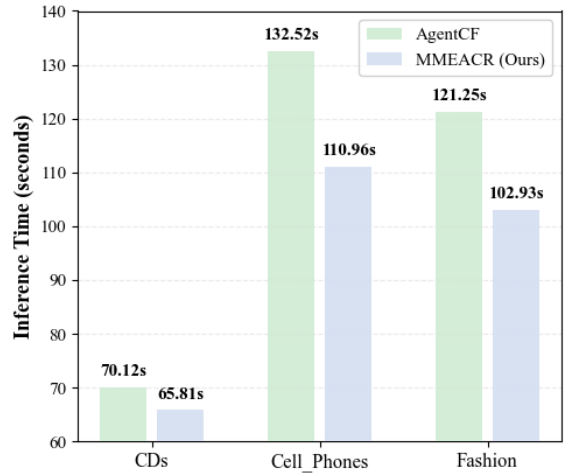


Figure 4: Inference time comparison between AgentCF and MMEACR on the three datasets. The Y-axis denotes the inference time (in seconds), while the X-axis denotes different datasets. MMEACR achieves faster inference across both datasets.

ration. To evaluate the computational efficiency of our proposed framework, we compare the inference time of MMEACR against the representative agent-based baseline, AgentCF, across three datasets. As illustrated in Figure 4, MMEACR consistently achieves lower inference latency than AgentCF in all evaluated domains. Specifically, MMEACR reduces inference time by 6.15% (from 70.12s to 65.81s) on CDs, 16.27% (from 132.52s to 110.96s) on Cell_Phones, and 15.11% (from 121.25s to 102.93s) on the Fashion dataset. These results demonstrate that while MMEACR incorpo-

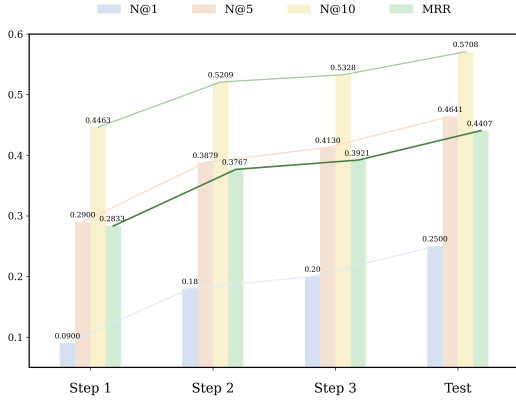


Figure 5: Performance evolution on Fashion. MMEACR improves steadily across optimization steps and achieves the best results at test time.

rates structured multimodal memory and reflective collaboration to achieve superior ranking performance (as shown in Table 1), it simultaneously optimizes the inference pipeline, proving its high efficiency and practicality for real-world recommendation scenarios.

Qualitative Analysis. As shown in Figure 3, while AgentCF generates coarse-grained and repetitive interest summaries (e.g., general “jazz” or “phone accessories”), MMEACR precisely captures fine-grained, specific user preferences (highlighted in red) such as a “cohesive and relaxing mood” or “heavy-duty protection with stylish design.” Furthermore, while AgentCF is vulnerable to interaction noise and suffers from preference drift by Round-5, MMEACR filters cross-modal redundancy via attribute-guided reflection, anchoring user profiles onto stable semantic blocks. This qualitative alignment corroborates our quantitative superiority, visually demonstrating that MMEACR achieves precise long-term profiling through attribute-level memory refinement.

Ablation Study. To evaluate the contribution of each component in our MMEACR framework, we conduct ablation studies by comparing the full **Multi-AgentCF** model (with both User Agent and Item Agent) against three simplified variants: **w/o Auto. Interaction**, which removes the automated interaction mechanism between agents and disables iterative mutual refinement; **w/o User Agent**, which excludes the User Agent and relies solely on item-side modeling; and **w/o Item Agent**, which removes the Item Agent and retains only user-centric reasoning. Table 3 reports results on both cell_phones and fashion domains. The full Multi-AgentCF consistently achieves the best performance across most metrics, demonstrating the

effectiveness of jointly modeling user and item agents within an interactive framework. In contrast, removing either agent leads to clear performance degradation, indicating that both user-side preference modeling and item-side contextual reasoning are essential for accurate ranking. Notably, the largest drop is observed when Auto. Interaction is removed, highlighting the importance of iterative inter-agent communication for refining memory and aligning preferences. These results verify that each component contributes to the overall performance and that their synergistic interaction is crucial for optimal recommendation quality.

Effectiveness of Iterative Memory Evolution.

To investigate how iterative memory refinement enhances recommendation performance, we analyze the evolution of key metrics throughout the optimization process on the Fashion dataset. Specifically, we report results at three consecutive optimization steps and the final testing phase to capture how progressive agent collaboration affects ranking quality over time. As illustrated in Figure 5, all evaluation metrics, including $N@1$, $N@5$, $N@10$, and MRR , exhibit a consistent upward trend as iterative interaction between the user and item agents proceeds. In particular, the $N@10$ score steadily increases from 0.4463 at Step 1 to 0.5708 at the final stage, while MRR improves from 0.2833 to 0.4407, with similar gains observed for $N@1$ and $N@5$, indicating consistent improvements across different ranking depths. These results suggest that iterative memory refinement enables continuous correction and enrichment of both user preference modeling and item contextual understanding, allowing the system to progressively accumulate more reliable reasoning traces and better align multimodal representations. Overall, the observed performance trajectory validates the effectiveness of our iterative update strategy and highlights the importance of continuous agent collaboration in improving recommendation quality.

6 Conclusion

In this paper, we propose MMEACR, a multimodal memory-enhanced agent collaboration framework for recommendation that overcomes modal-lacking limitations in text-only LLM-based systems by jointly leveraging visual and textual features and enabling dynamic memory evolution through iterative agent interactions. By incorporating a dedicated multimodal embedding memory and RRF-

based ranking fusion, MMEACR produces more accurate and interpretable recommendations. Extensive experiments on real-world datasets show that MMEACR surpasses strong baselines, particularly in capturing nuanced user preferences from multimodal signals, and ablation studies further validate the contribution of each individual component. In addition, qualitative analyses demonstrate that the proposed agent collaboration mechanism can generate more consistent and preference-aligned reasoning across interaction steps. Overall, our results highlight the effectiveness of integrating multimodal memory with iterative agent collaboration for improving recommendation quality in LLM-based systems.

Limitations

While MMEACR demonstrates strong performance and efficient inference, one limitation is the long-term scalability of the continuous memory evolution. As user interactions scale over extended periods, the iterative accumulation in the multimodal embedding memory could potentially increase storage and retrieval costs. Although this is mitigated by our RRF-based ranking fusion in current benchmarks, optimizing the balance between memory capacity and lifelong efficiency remains a promising direction for our future work.

References

- Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, pages 1007–1014. ACM.
- Mohamed Amine Ferrag, Norbert Tihanyi, and Mérouane Debbah. 2025. From LLM reasoning to autonomous AI agents: A comprehensive review. *arXiv preprint arXiv:2504.19678*.
- Tianqi Gao, Chengkai Huang, Zihan Wang, Cao Liu, Ke Zeng, and Lina Yao. 2026. Factorized latent reasoning for llm-based recommendation. *arXiv preprint arXiv:2604.26760*.
- Jesse Harte, Wouter Zorgdrager, Panos Louridas, Asterios Katsifodimos, Dietmar Jannach, and Marios Fragkoulis. 2023. Leveraging large language models for sequential recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, pages 1096–1102.
- Yupeng Hou, Junjie Zhang, Zhipeng Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024a. Large language models are zero-shot rankers for recommender systems. In *Proceedings of the European Conference on Information Retrieval (ECIR)*, pages 364–381, Cham. Springer Nature Switzerland.
- Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024b. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval*, pages 364–381.
- Chengkai Huang, Hongtao Huang, Tong Yu, Kaige Xie, Junda Wu, Shuai Zhang, Julian McAuley, Dietmar Jannach, and Lina Yao. 2025a. A survey of foundation model-powered recommender systems: From feature-based, generative to agentic paradigms. *arXiv preprint arXiv:2504.16420*.
- Chengkai Huang, Shoujin Wang, Xianzhi Wang, and Lina Yao. 2023a. Dual contrastive transformer for hierarchical preference modeling in sequential recommendation. In *Proceedings of the 46th international acm sigir conference on research and development in information retrieval*, pages 99–109.
- Chengkai Huang, Shoujin Wang, Xianzhi Wang, and Lina Yao. 2023b. Modeling temporal positive and negative excitation for sequential recommendation. In *Proceedings of the ACM Web Conference 2023*, pages 1252–1263.
- Chengkai Huang, Junda Wu, Yu Xia, Zixu Yu, Ruhan Wang, Tong Yu, Ruiyi Zhang, Ryan A Rossi, Branislav Kveton, Dongruo Zhou, and 1 others. 2025b. Towards agentic recommender systems in the era of multimodal large language models. *arXiv preprint arXiv:2503.16734*.
- Chengkai Huang, Tong Yu, Kaige Xie, Shuai Zhang, Lina Yao, and Julian McAuley. 2024a. Foundation models for recommender systems: A survey and new perspectives. *arXiv preprint arXiv:2402.11143*.
- Hongtao Huang, Chengkai Huang, Tong Yu, Xiaojun Chang, Wen Hu, Julian McAuley, and Lina Yao. 2024b. Dual conditional diffusion models for sequential recommendation. *arXiv preprint arXiv:2410.21967*.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.
- Shuguang Jiao, Chengkai Huang, Shuhan Qi, Xuan Wang, Yifan Li, and Lina Yao. 2026a. Doctor-rag: Failure-aware repair for agentic retrieval-augmented generation. *arXiv e-prints*, pages arXiv–2604.
- Shuguang Jiao, Xinyu Xiao, Yunfan Wei, Shuhan Qi, Chengkai Huang, Quan Z Sheng, and Lina Yao. 2026b. Prunerag: Confidence-guided query decomposition trees for efficient retrieval-augmented generation. In *Proceedings of the ACM Web Conference 2026*, pages 1923–1934.

- Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE.
- Yifan Li, Ismail Jafarov, Kenan Zeynalov, and Chen Tang. 2025. CLEAR-Rec: A contrastive long-term memory-enhanced, explainable, and adaptive recommendation framework. In *Proceedings of the 2nd International Conference on Artificial Intelligence of Things and Computing*, pages 236–244. ACM.
- Jiahao Liu, Shengkang Gu, Dongsheng Li, Guangping Zhang, Mingzhe Han, Hansu Gu, Peng Zhang, Tun Lu, Li Shang, and Ning Gu. 2025. Agentcf++: Memory-enhanced llm-based agents for popularity-aware cross-domain recommendations. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2566–2571. ACM.
- Chenyang Lyu, Minghao Wu, Longyue Wang, Xinyu Huang, Binhang Liu, Z those Du, Ziyu Min, Hui Su, Dengqun Bi, Guan jie Pan, , and 1 others. 2023. Macaw-LLM: Multi-modal language modeling with image, audio, video, and text integration. *arXiv preprint arXiv:2306.09093*.
- Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Anuja Salve, Shreyas Attar, Mansi Deshmukh, Satej Shivpuje, and Aashish Manik Utsab. 2024. A collaborative multi-agent approach to retrieval-augmented generation across diverse data. *arXiv preprint arXiv:2412.05838*.
- Tian Shi, Jing Xu, Xin Zhang, Xiaolei Zang, Ke Zheng, Yue Song, and Hang Li. 2025. Retrieval augmented generation with collaborative filtering for personalized text generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1294–1304. ACM.
- Zhenheng Tang, Xin He, Tiancheng Zhao, Fanjunduo Wei Wei, Xiang Liu, Peijie Dong, Qian Wang, Zehao Li, Xiaowen Chu, , and 1 others. 2026. Llm agent memory: A survey from a unified representation–management perspective. *arXiv preprint arXiv:2603.03590*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.
- Juexiang Ye, Xue Li, Yang Xinyu, Chengkai Huang, Lanshun Nie, Lina Yao, and Dechen Zhan. 2026. Memweaver: Weaving hybrid memories for traceable long-horizon agentic reasoning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 12928–12956.
- Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian J. McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024. Agentcf: Collaborative learning with autonomous language agents for recommender systems. In *Proceedings of the ACM on Web Conference*, pages 3679–3689. ACM.
- Xin Zhang, Yanzhao Zhang, Wen Xie, Mingxin Li, Ziqi Dai, Dingkun Long, Pengjun Xie, Meishan Zhang, Wenjie Li, and Min Zhang. 2025a. [Gme: Improving universal multimodal retrieval by multimodal llms](#). *Preprint*, arXiv:2412.16855.
- Zhen Zhang, Quanyu Dai, Xiangbo Bo, Chen Ma, Ronghua Li, Xu Chen, Ruiming Zhang, Ji-Rong Wen, , and 1 others. 2025b. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems (TOIS)*, 43(6):1–47.
- Zhe Zhao, Wenqi Fan, Jiatong Li, Xiao-Yu Liu, Xiao Mei, Yanan Wang, Qing Li, and 1 others. 2024. Recommender systems in the era of large language models (LLMs). *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 36(11):6889–6907.

A Computational Cost and Efficiency

Due to the high cost of LLM API calls, we sample 100 users and their historical interactions per dataset, leading to about 500 interaction steps across 5 training rounds. We use a tiered model strategy, where GPT-4o (Hurst et al., 2024) is applied for memory initialization, interaction simulation, and inference-time ranking, balancing reasoning quality with efficiency and supporting asynchronous batch processing (batch size 4 for training, 5 for evaluation).

At inference, description-based ranking (DES) prompts GPT-4o with the user’s natural-language memory profile and 10 candidate item descriptions to directly output a ranked list, which is mapped back to item IDs using fuzzy matching. In parallel, embedding-based ranking (EMB) encodes user memory and multimodal item content into dense vectors and ranks candidates via cosine similarity without LLM inference. The final result is obtained by fusing DES and EMB rankings using Reciprocal Rank Fusion, with a fallback to EMB-only when DES outputs are invalid.

Overall, inference cost is dominated by a single LLM call per user, and full evaluation remains efficient, completing within about 100 seconds per dataset and showing a 6–16% speedup over AgentCF due to the streamlined memory design and shorter prompts.

B Algorithm

Algorithm 1: Dual-Stage Attribute-Guided Memory Update (UAMG)

```

Input : Turn  $t \in \{1, \dots, T\}$ ; positive title  $s^+$ ,
        negative title  $s^-$ ; prior user memory  $U^{(t-1)}$ ;
        item memories  $M^+, M^-$ .
Output: Updated memories  $U^{(t)}, M^{+(t)}, M^{-(t)}$ .

// Stage 1: Agent decision
1  $P_{ch} \leftarrow$ 
   ChoicePrompt( $U^{(t-1)}, s^+, s^-, M^+, M^-$ )
2  $(\hat{s}, r) \leftarrow \text{LLM}(P_{ch})$  //  $\hat{s}$ : chosen title;  $r$ :
   rationale
3  $y \leftarrow \mathbb{1}[\text{Match}(\hat{s}, s^+) > \text{Match}(\hat{s}, s^-)]$  //  $y = 1$ :
   correct

// Stage 2: Attribute extraction
4 if  $y = 1$  then
5    $A \leftarrow$ 
     LLM(Attr $_{+}(U^{(t-1)}, s^+, s^-, M^+, M^-, r)$ )
6 else
7    $A \leftarrow$ 
     LLM(Attr $_{-}(U^{(t-1)}, s^+, s^-, M^+, M^-, r)$ )
8 end

// Stage 3: Memory update
9  $(P_u, P_i) \leftarrow$ 
   UpdatePrompt( $U^{(t-1)}, M^+, M^-, s^+, s^-, r, A, y$ )

10  $\tilde{U}, \tilde{M}^+, \tilde{M}^- \leftarrow \text{LLM}(P_u), \text{LLM}(P_i)$ 
11 return  $U^{(t)}, M^{+(t)}, M^{-(t)}$ 

```

C Prompt Templates

Attribute Extraction Templates (Correct Version)

Use: Attribute extraction system for user preference analysis. No LLM is used to generate these queries. Each template wraps the deterministic user preferences, item details, and reasoning.

Variables:

- {user_description}: User preferences description.
- {pos_item_title}: Title of the correct item matching user preference.
- {neg_item_title}: Title of the rejected item.
- {system_reason}: Previous system reasoning or explanation.
- {attr_list}: Allowed attribute dimensions joined by commas.

Templates:

1. You are an attribute extraction system. Your ONLY task is to extract attributes from the allowed list ({attr_list}) that explain WHY the correct item ({pos_item_title}) matches the user preference: {user_description}.
2. Given the correct item {pos_item_title} and the rejected item {neg_item_title}, extract at least 2 attributes in the required format based on the user's needs: {user_description}.
3. Analyze the previous reasoning ({system_reason}) to structure the final answer. Follow STRICT RULES: no explanations, no reasoning, no self-introduction, no markdown, no summaries, and do NOT invent attributes.

Figure 6: The structured query wrapper template containing rules, formatting, and variables for correct attribute analysis and extraction.

Attribute Extraction Templates (Incorrect Version)

Use: LLM-based prompt templates for simultaneous positive/negative attribute extraction. This setup forces the model to perform complex evaluation, scoring, and extreme negative filtering within a single completion step.

Variables:

- {user_description}: User preferences description.
- {pos_item_title}: Title of the correct item matching user preference.
- {neg_item_title}: Title of the rejected item.
- {system_reason}: Previous system reasoning or explanation.
- {attr_list}: Allowed attribute dimensions joined by commas.

Templates:

1. You are an attribute extraction system. Your ONLY task is to extract positive attributes from {pos_item_title} and misleading negative attributes from {neg_item_title} that explain the user preference: {user_description}. Allowed attributes ONLY: {attr_list}.
2. Given the correct item {pos_item_title} and the rejected item {neg_item_title}, extract at least 2 attributes in the required format based on the user's needs: {user_description}.
3. Analyze the previous reasoning ({system_reason}) to structure the final answer. Follow STRICT RULES: no explanations, no reasoning, no self-introduction, no markdown, no summaries, and do NOT invent attributes.

Figure 7: The structured query wrapper template containing rules, formatting, and variables for incorrect attribute analysis and extraction.

User Memory Reflection with attribute guidance Template (Correct Version)

Use: LLM-based prompt template for validating correct preference judgments through attribute-level rationale extraction and personalized self-introduction refinement. The template encourages the model to reinforce accurate preferences and consolidate stable user characteristics.

Variables:

- {list_of_item_description}: Descriptions of the compared candidate items.
- {pos_item_title}: Title of the correctly selected item preferred by the user.
- {neg_item_title}: Title of the rejected item disliked by the user.
- {system_reason}: User's previous reasoning or explanation for the successful choice.
- {attribute_dimensions}: Allowed attribute dimensions used for attribute-level preference analysis.

Templates:

1. Recently, you compared two candidate items described in {list_of_item_description}. You selected {pos_item_title} and rejected {neg_item_title}. Your explanation was: {system_reason}.
2. After encountering these items, you confirmed that you truly prefer {pos_item_title} and dislike {neg_item_title}. This indicates your original judgment about your preferences was accurate.
3. Perform attribute-level rationale extraction using ONLY the allowed dimensions: {attribute_dimensions}. Identify 1–3 key attributes that contributed to this successful match. For each attribute, specify:
 - the related item,
 - whether the attribute is positive or negative to the preference,
 - and an importance score from 1 to 5.
4. Based on the extracted attributes, analyze and reinforce:
 - confirmed preferences from {pos_item_title},
 - confirmed dislikes from {neg_item_title},
 - and stable preference patterns reflected in the previous reasoning.
5. Generate an updated self-introduction by:
 - first emphasizing newly confirmed preferences,
 - then summarizing consistent past preferences,
 - and finally describing dislikes.
6. Follow STRICT RULES:
 - Output MUST strictly follow the required structure.
 - The self-introduction must be under 150 words.
 - Be concise, specific, and personalized.
 - Do NOT include reasoning processes in the self-introduction.
 - Avoid generic preference descriptions.

Figure 8: The structured prompt template for successful preference confirmation and personalized self-introduction refinement.

User Memory Reflection with attribute guidance Template (Incorrect Version)

Use: LLM-based prompt template for correcting mistaken preference judgments through attribute-level reflection and self-introduction updating. The template guides the model to analyze misconception attributes, identify true preferences/dislikes, and refine personalized user profiles.

Variables:

- {list_of_item_description}: Descriptions of the compared candidate items.
- {pos_item_title}: Title of the finally preferred (correct) item.
- {neg_item_title}: Title of the mistakenly chosen and later rejected item.
- {system_reason}: User's previous reasoning or explanation for the incorrect choice.
- {attribute_dimensions}: Allowed attribute dimensions used for attribute-level analysis.

Templates:

1. Recently, you compared two candidate items described in {list_of_item_description}. Although you initially selected {neg_item_title}, you later realized that {pos_item_title} better matches your true preferences. Your previous reasoning was: {system_reason}.
2. Perform attribute-level misconception analysis using ONLY the allowed dimensions: {attribute_dimensions}. Identify 1–3 key attributes that caused the mistaken judgment. For each attribute, specify:
 - the related item,
 - whether the attribute is positive or negative to the true preference,
 - and an importance score from 1 to 5.
3. Based on the extracted attributes, analyze why the previous judgment was incorrect and identify:
 - newly discovered preferences from {pos_item_title},
 - newly identified dislikes from {neg_item_title},
 - and conflicts between old and updated preferences.
4. Generate an updated self-introduction by:
 - first emphasizing new preferences,
 - then summarizing consistent past preferences,
 - and finally describing dislikes.
5. Follow STRICT RULES:
 - Output MUST strictly follow the required structure.
 - The self-introduction must be under 150 words.
 - Be concise, specific, and personalized.
 - Do NOT include reasoning processes in the self-introduction.
 - Avoid generic preference descriptions.

Figure 9: The structured prompt template for attribute-level misconception correction and personalized self-introduction updating.

User Memory Reflection without attribute guidance Template (Correct Version)

Use: LLM-based prompt template for reinforcing correct preference judgments without explicit attribute-level guidance. The template helps the model consolidate accurate user preferences and refine personalized self-introductions based on successful interaction outcomes.

Variables:

- {list_of_item_description}: Descriptions of the compared candidate items.
- {pos_item_title}: Title of the correctly selected and preferred item.
- {neg_item_title}: Title of the rejected and disliked item.
- {system_reason}: User's previous reasoning or explanation for the successful choice.

Templates:

1. Recently, you compared two candidate items described in {list_of_item_description}. You selected {pos_item_title} and rejected {neg_item_title}. Your previous reasoning was: {system_reason}.
2. After encountering these items, you confirmed that you truly prefer {pos_item_title} and dislike {neg_item_title}. This indicates your original judgment about your preferences was accurate.
3. Analyze your previous reasoning and identify:
 - preference patterns correctly reflected in your earlier judgment,
 - newly confirmed preferences from {pos_item_title},
 - and dislikes associated with {neg_item_title}.
4. Summarize your previous preferences and dislikes, combining them with newly confirmed insights while removing inconsistent or conflicting parts.
5. Generate an updated self-introduction by:
 - first emphasizing newly confirmed preferences,
 - then summarizing consistent past preferences,
 - and finally describing dislikes.
6. Follow STRICT RULES:
 - Output format MUST be: My updated self-introduction: [Your updated self-introduction here].
 - The self-introduction must be under 150 words.
 - Be concise, clear, and personalized.
 - Do NOT describe reasoning processes in the self-introduction.
 - Describe only preferred or disliked item characteristics.
 - Avoid generic preference descriptions.

Figure 10: The structured prompt template for successful preference confirmation and self-introduction refinement without explicit attribute-level guidance.

User Memory Reflection without attribute guidance Template (Incorrect Version)

Use: LLM-based prompt template for correcting mistaken preference judgments without explicit attribute-level guidance. The template directly guides the model to revise preference understanding and update personalized self-introductions based on corrected user experiences.

Variables:

- `{list_of_item_description}`: Descriptions of the compared candidate items.
- `{pos_item_title}`: Title of the finally preferred (correct) item.
- `{neg_item_title}`: Title of the mistakenly selected and later disliked item.
- `{system_reason}`: User's previous reasoning or explanation for the incorrect choice.

Templates:

1. Recently, you compared two candidate items described in `{list_of_item_description}`. Although you initially selected `{neg_item_title}`, you later realized that `{pos_item_title}` better matches your true preferences. Your previous reasoning was: `{system_reason}`.
2. Analyze the misconceptions in your previous judgment and explain how your understanding of your preferences has changed after encountering these items.
3. Identify:
 - new preferences discovered from `{pos_item_title}`,
 - new dislikes identified from `{neg_item_title}`,
 - and conflicts between old and updated preferences.
4. Summarize your previous preferences and merge them with the newly discovered insights while removing inconsistent or conflicting parts.
5. Generate an updated self-introduction by:
 - first emphasizing new preferences,
 - then summarizing past consistent preferences,
 - and finally describing dislikes.
6. Follow STRICT RULES:
 - Output format MUST be: My updated self-introduction: [Your updated self-introduction here].
 - The self-introduction must be under 150 words.
 - Be concise, clear, and personalized.
 - Do NOT describe reasoning processes in the self-introduction.
 - Describe only preferred or disliked item characteristics.
 - Avoid generic preference descriptions.

Figure 11: The structured prompt template for preference correction and self-introduction updating without explicit attribute-level guidance.